FiHaven — Information Security Policy

Owner	Daniel Hipskind (FiHaven) — acting Security Officer
Applies to	FiHaven web app, API/backend, iOS app, Android app, and supporting infrastructure (collectively, the "Service")
Security contact	security@fihaven.app
Version	1.0
Effective date	2026-06-08
Review cadence	Reviewed at least annually and upon any material change to the architecture, data flows, or third-party processors

> This document is FiHaven's documented information security policy and the procedures by which it is operationalized to **identify, mitigate, and monitor** information security risks relevant to the business. It complements the vulnerability-disclosure policy in `.github/SECURITY.md`.

---

1. Purpose & scope

FiHaven is a personal bill, credit-card, and debt-payoff dashboard. Users hold real accounts with server-side sync across web, iOS, and Android. This policy defines how FiHaven protects the confidentiality, integrity, and availability of:

- **User account data** — email, display name, authentication credentials, MFA secrets.
- **User financial data** — bills, credit cards, budgets, payment history (all user-entered).
- **Connected-account data** — balances and account metadata retrieved via Plaid, and the Plaid **access tokens** used to retrieve them.
- **The systems** that store and process the above.

It applies to all FiHaven environments (development, staging where applicable, and production) and to anyone with access to FiHaven systems, source code, or production data.

---

2. Roles & responsibilities (governance)

FiHaven is operated by a small team. The **Security Officer** (currently the owner) is accountable for this program: maintaining this policy, owning risk decisions, approving access, and coordinating incident response. Anyone granted access to FiHaven systems is responsible for adhering to this policy and reporting suspected security issues to the security contact above.

Security is a standing consideration in design and code review, not a separate gate; material changes that affect data handling, authentication, or third-party data flows require explicit Security Officer review before release.

---

3. Risk management — identify, mitigate, monitor

FiHaven runs a continuous, lightweight risk-management cycle:

Identify. Risks are identified through (a) threat modeling of new features that touch authentication, payments, or connected-account data; (b) automated static analysis (GitHub CodeQL) on every push and pull request; (c) automated dependency scanning

(Dependabot + dependency review) for known-vulnerable libraries; (d) GitHub secret scanning; and (e) review of provider security bulletins (Plaid, Stripe, Cloudflare, hosting, runtime/OS).

Mitigate. Identified risks are tracked and remediated based on severity. Controls are layered: encryption in transit and at rest, least-privilege access, MFA, input validation, rate limiting, bot mitigation, CSRF protection, and dependency patching. High-severity issues are prioritized for immediate remediation; lower-severity issues are scheduled and tracked to closure.

Monitor. The Service is monitored through application logs, process-manager logs (PM2), and reverse-proxy access/error logs (nginx). Automated CI re-runs security analysis on every change. Authentication anomalies (e.g., repeated failed logins) are constrained by rate limiting. The risk posture and this policy are reviewed at least annually.

A risk that cannot be immediately remediated is documented with a compensating control and a remediation target date, approved by the Security Officer.

---

4. Data classification & handling

Class	Examples	Handling
Restricted	Plaid access tokens, MFA secrets/TOTP seeds, password hashes, encryption keys	Encrypted at rest (AES-256-GCM) or one-way hashed; never logged; never returned to clients; never committed to version control
Confidential	User financial data, connected-account balances, email addresses	Access-controlled per user; transmitted only over TLS; retained only while the account exists
Internal	Application logs, operational metrics	Access limited to operators; no Restricted data permitted in logs
Public	Marketing pages, open-source code	No protection beyond integrity

Restricted and Confidential data are only ever transmitted over encrypted channels and are scoped to the owning user by server-side authorization on every request.

---

5. Access control & authentication

- **End-user authentication.** Passwords are hashed with **bcrypt** (configurable cost). Sessions are short-lived and bound server-side; web sessions use `Secure`, `HttpOnly`, `SameSite` cookies with a per-session **CSRF token**, and native apps use bearer tokens stored in the platform secure store (iOS **Keychain**, Android **EncryptedSharedPreferences**).
- **Multi-factor authentication.** Users may enroll TOTP authenticator apps, **WebAuthn passkeys**, and/or email one-time codes. MFA secrets are encrypted at rest (§6).
- **Bot & abuse mitigation.** Sign-up and sign-in are protected by **Cloudflare Turnstile** and server-side **rate limiting** (per-IP and per-email).
- **Administrative/operator access.** Production access (server SSH, hosting console, third-party dashboards) is restricted to the Security Officer, protected by strong unique credentials and MFA on every provider that supports it, and granted on a least-privilege, need-to-know basis. Access is reviewed when roles change and revoked promptly when no longer required.
- **Authorization.** Every data and account API enforces server-side authorization so a session can only read or modify its own user's data; sensitive routes additionally require a verified email and, for billing/bank features, an active entitlement.

---

6. Encryption & key management

- **In transit.** All client–server traffic is served over **HTTPS/TLS**; the reverse proxy terminates TLS and the application sets the `Secure` flag on session cookies. Plaintext HTTP is not used for authenticated traffic.

- **At rest.** Secrets classified as Restricted — **Plaid access tokens** and **MFA/TOTP secrets** — are encrypted with **AES-256-GCM** using a unique random 12-byte IV per record and an authentication tag, via a single vetted encryption helper shared across the codebase.
- **Key management.** The 256-bit at-rest encryption key is supplied via an environment variable (`MFA_ENCRYPTION_KEY`) or, if absent, generated once and persisted to a key file with restrictive (`0600`) permissions outside the web root. Keys and all other secrets live only in environment configuration (`.env`), which is excluded from version control and stripped/sanitized during deployment. Plaid client secrets, Stripe keys, and SMTP credentials are handled the same way.
- **No secrets in code.** Source control is scanned for secrets; credentials, tokens, and keys are never committed.

---

7. Application & secure development (SDLC)

- Source is managed in Git with change history and code review via pull requests on the mainline branch.
- **Continuous integration** builds and tests the web, iOS, and Android targets on every push and pull request.
- **Static analysis** (GitHub CodeQL) runs on JavaScript/TypeScript, Java/Kotlin, and Swift.
- **Dependency hygiene** is automated with Dependabot (updates) and dependency review (blocks introduction of known-vulnerable packages); `npm audit` is used for the backend.
- Input is validated and output encoded to defend against injection and XSS; database access uses parameterized/prepared statements exclusively.
- Authentication, payments, and connected-account changes receive Security Officer review before merge.

---

8. Infrastructure & network security

- The Service runs as a single deployable Node.js/Express application backed by SQLite, on a hardened **Linux VPS**, managed by the **PM2** process manager behind an **nginx** reverse proxy that terminates TLS.
- The application trusts only the first proxy hop and applies standard security headers.
- The host is kept patched; only required ports are exposed; the outbound mail service is bound to loopback.
- Production data (the SQLite database and the encryption key file) is persisted outside the deployment artifact and is never copied into source control or build outputs.

---

9. Vulnerability & patch management

FiHaven runs a defined vulnerability-management program spanning source code, dependencies, production hosts, and operator endpoints:

- **Source code** is analyzed on every push and pull request (GitHub CodeQL); findings are triaged by severity.
- **Dependencies** are monitored continuously (Dependabot) and gated against known-vulnerable packages (dependency review); `npm audit` supplements the backend.
- **Production hosts** receive operating-system and runtime security patches on a regular cadence; available security updates are surfaced by the host package manager and applied promptly.
- **Operator endpoints** (workstations used to administer the Service) run the platform's built-in malware/vulnerability protection and automatic security updates, with full-disk encryption and screen-lock enforced.
- **End-of-life (EOL) software** is actively avoided and tracked: the runtime (Node.js) and key dependencies are kept on supported versions, and EOL components are scheduled for replacement before end of support.
- **Remediation SLAs** (targets, measured from confirmation): **Critical — 7 days; High — 30 days; Medium — 90 days; Low — next scheduled maintenance.** Any issue that cannot meet its SLA is documented with a compensating control and an approved exception (§18).

---

10. Logging, monitoring & alerting

- Application, process-manager, and reverse-proxy logs are retained for operational and security review. Logs **must not** contain Restricted data (no access tokens, secrets, passwords, or full PANs — FiHaven never handles card numbers; see §13).
- Authentication failures are rate-limited and observable.
- CI security analysis acts as continuous monitoring of the codebase; provider dashboards (Plaid, Stripe, Cloudflare) are reviewed for anomalies and alerts.

---

11. Incident response

If a security incident is suspected (unauthorized access, data exposure, credential compromise, or abuse of the Plaid integration):

1. **Detect & triage** — confirm the issue, assess scope and severity, and preserve relevant logs/evidence.
2. **Contain** — revoke affected sessions and credentials, rotate keys/secrets as needed, and disable the affected capability if necessary. *Changing a user's password already invalidates that user's other sessions; operator credentials and encryption keys are rotated on suspicion of compromise.*
3. **Eradicate & recover** — remediate the root cause, restore from backups if integrity is in question, and verify the fix.
4. **Notify** — notify affected users and relevant providers/regulators as required by the situation and applicable law. **Plaid is notified without undue delay of any incident affecting Plaid data or access tokens**, per our agreement with Plaid.
5. **Review** — conduct a post-incident review and update controls and this policy.

Vulnerability reports from external researchers are handled per `.github/SECURITY.md`.

---

12. Backup & business continuity

- The SQLite database and the encryption key are persisted across deployments and backed up. Backups are protected at the same classification as the source data.
- Because the application is a single self-contained deployable, recovery consists of restoring the database/key and redeploying the application.
- Restore procedures are validated periodically.

---

13. Third-party / vendor management

FiHaven relies on a small number of vetted processors, each governed by its agreement and used for a single purpose:

Vendor	Purpose	Data shared	Notes
Plaid	Optional bank-account balance retrieval	User-initiated bank link; FiHaven holds an encrypted access token	See §14
Stripe	Subscription billing (FiHaven Pro)	Payment details handled entirely by Stripe	**FiHaven never receives or stores card numbers**; PCI-DSS scope is borne by Stripe
Cloudflare	Bot mitigation (Turnstile)	Challenge token only	No financial data
VPS host (Hostinger)	Compute/storage for the Service	Encrypted-at-rest data on the host	Hardened, patched, access-restricted

Vendor	Purpose	Data shared	Notes
Email (self-hosted Postfix/OpenDKIM or SMTP relay)	Transactional email (verification, reset, reminders)	Email address + message content	SPF/DKIM/DMARC aligned

New processors are reviewed for their security posture before adoption; data shared with each is minimized to what the function requires.

---

14. Plaid data handling

Bank linking via Plaid is an **optional, Pro-gated convenience overlay** — FiHaven is fully usable with manually entered data, so a dropped or absent Plaid connection never breaks a user's dashboard. The following controls govern Plaid data specifically:

- **User-initiated, consent-based.** Links are only ever created by the user through Plaid Link (web Link SDK, iOS LinkKit, Android Link SDK). FiHaven never asks for or handles bank login credentials directly.
- **Server-issued tokens.** Plaid **link tokens** are minted server-side and are short-lived. The Plaid `client_id` / `secret` are never exposed to clients.
- **Access tokens are treated as bank credentials.** The `public_token` returned by Link is exchanged **server-side** for an access token, which is immediately **AES-256-GCM encrypted** before storage (§6), is **never returned to any client**, is **never written to logs**, and is **never committed to source control**.
- **Least data / scope minimization.** Only the Plaid products required for the feature are requested (configured via `PLAID_PRODUCTS`; balances are the surfaced data). FiHaven retrieves the minimum necessary to display balances to the owning user.
- **Data isolation.** Connected-account data is scoped to the owning user by server-side authorization on every request and is used solely to display balances within that user's own dashboard. **Plaid data is never sold, rented, or shared** with third parties.
- **User control & deletion.** Users can disconnect a linked institution at any time; disconnecting **removes the stored item and its encrypted access token**. Deleting the FiHaven account removes all associated Plaid items and tokens.
- **Environment separation.** Sandbox and production are separated via `PLAID_ENV`, each with its own credentials; production credentials are used only in production.
- **Webhooks.** Plaid webhooks are received on a dedicated endpoint for item/transaction status; webhook handling does not weaken authentication on user-facing routes.
- **Incident notification.** Any incident affecting Plaid data or access tokens triggers prompt notification to Plaid (§11).

---

15. Data retention & deletion

- User and financial data are retained while the account is active and deleted when the user deletes their account.
- Plaid items and encrypted access tokens are deleted on disconnect or account deletion (§14).
- Single-use, expiring tokens (email verification, password reset, 2FA recovery) are SHA-256-hashed at rest and invalidated on use or expiry.
- Backups are aged out on a defined cycle; Restricted data is not retained longer than necessary for the function it serves.

---

16. Personnel & acceptable use

Anyone with access to FiHaven systems must: use strong, unique credentials with MFA; access production data only as needed to operate the Service; never copy Restricted data to unmanaged devices or third-party tools; never disable security controls without Security Officer approval; and report suspected incidents immediately. Access is provisioned on least-privilege and revoked when no longer needed.

17. Physical security

FiHaven operates no on-premises production infrastructure. Production compute and storage are hosted by the VPS provider, which maintains physical and environmental controls for its data centers. Operator workstations used to administer the Service are kept patched, full-disk-encrypted, and protected by screen lock and MFA.

---

18. Compliance, exceptions & review

- Adherence to this policy is mandatory for everyone in scope (§1).
- Exceptions require documented Security Officer approval, a compensating control, and a remediation date.
- This policy is reviewed at least annually and after any material change to architecture, data flows, or processors. The Security Officer maintains version history below.

Revision history

Version	Date	Change
1.0	2026-06-08	Initial documented information security policy.